

DIGITAL LOGIC AND SYSTEM DESIGN

LAB ASSIGNMENT 4A

AYUSH SINGHAL (2025CS51643)

LOKESH DAYAL (2025CS11991)

INTRODUCTION->

In this lab we designed a 8-bit hamming distance calculator. The Hamming distance between two equal-length binary words is defined as the number of bit positions at which they differ. We first computed XOR of two numbers as difference and counting no of ones in that difference was our main objective keeping in mind the constraints that $d(A,A) = 0$ for all A. $d(A,B) = d(B,A)$ and $0 \leq d(A,B) \leq 8$ (max possible sum of bits in 8 bit number).

MODULE DESCRIPTION AND IMPLEMENTATION->

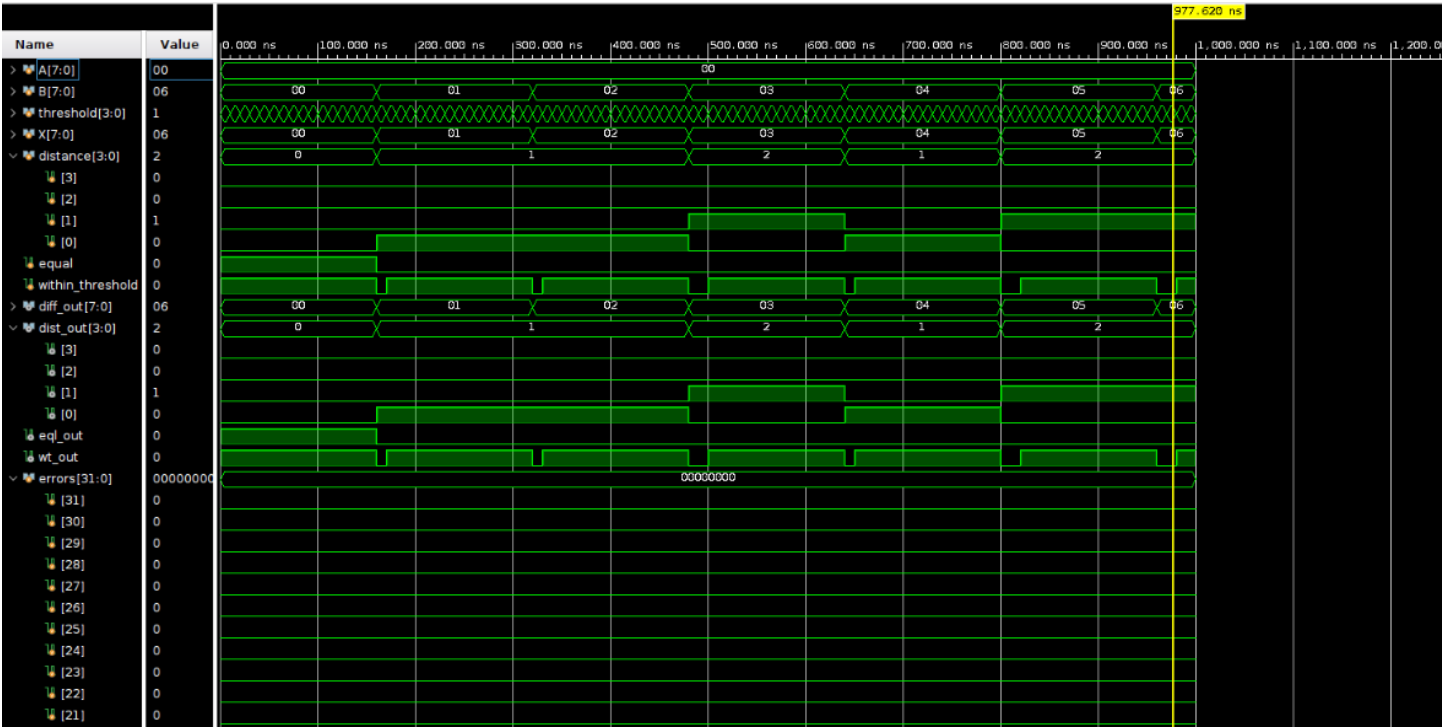
We designed the whole task by breaking it in different modules to reuse and optimize implemented subtasks keeping in mind that arithmetic operators are not allowed.

- 1) **HALF ADDER** – this was a standard half-adder taking in 2 bits and giving output as 2 bits one for the sum and other or carry strictly using XOR, AND only. It was a sub-module used in implementation of full_adder.
- 2) **FULL ADDER** - this was also a standard full adder taking in 2 bits and a carry as input summing the bits and carry to give the sum bit and carry bit as output strictly using XOR, AND, OR only. It is used in other implementations.
- 3) **FULL ADDER 2 BIT** - this module takes in two 2 bit numbers as input and outputs a 2 bit number along with a carry. It uses idea from the ripple-carry adder and is built by instantiating two full_adder modules.
- 4) **FULL ADDER 3 BIT** - this module takes in two 3 bit numbers as input and outputs a 3 bit number along with a carry. It also extends idea of the ripple-carry adder and is built by instantiating one full_adder2bit and one full_adder modules.
- 5) **HAMMING DISTANCE** - We have here implmeneted a hierarchical design going from top to bottom adding smaller first and then growing finally to compute $d(A,B)$. First the distance is computed by XOR and stored in a wire t' Then we add pairwise consecutive bits by 4 full adders giving us 4 2 bit

numbers which are in then added in pairs by 2 bit adders and finally two 3 bit numbers are added by a 3 bit number to output the 4 bit distance. Then this distance is ensured if it is within threshold or not. And finally checking if the given 2 numbers are equal or not.

- 6) **COMPARATOR_4BIT** – It takes in input as two 4 bit numbers a,b and outputs 1 if $a \leq b$ otherwise 0. it is implemented by a standard 4 bit comparator studied by us as given in textbook.
- 7) **COMPARATOR_8BIT** - It takes two 8 bit numbers and outputs if both the numbers are equal or not by using the reduce funcn(which is given in verilog slides given by mam) which drastically reduce the computation by taking AND of bits of NOT of the XOR of A and B.
- 8) **HAMMING TESTBENCH** - The hamming testbench is made for checking the correctness of the code by running a simulation. The idea for the testbench is as we have to check all possible configurations of A, B and threshold we use 3 for loops generating all possibilities which are $2^{(20)}$ in no. We kept an error variable for keeping a track of number of errors in the loops. Every possible config first computes outputs as computed by the design and then compare it with actual outputs which are simply computed by us using algebraic operators. Thus it checks A and B are equal or not, distance computed(already between 0 and 8) and outputted are same or not, distance is within threshold or not, XOR computed and outputted are same or not. If any error is present is it increments the error and as well as displays the error. If all the test cases pass and errors are 0 the code prints PASS as summary otherwise it prints the summary FAILED,how many errors occurred.

WAVEFORMS OBTAINED



BEHAVIORAL SIMULATION WAVEFORM

PASSED

TCS –CONSOLE -OUTPUT

DEBUGGING CHALLENGES

We first made a mistake in 2 bit- we mixed up 2 bit adder and full adder which was realised by us after a while. Then obviously there were some syntax errors in whole project which was corrected by us. During writing testbench we did a blunder by inheriting the module inside the for loop . And finally while debugging after running testbench we found the most frustrating mistake in hamming.v we always added bit 0 and bit 1 in all the 2 full adders giving us high no of errors in simulation.

THANKYOU !